LD1-B Production Atomic Block-Sparse Density Specification

Periodic block packing, atomic preparation, resource preflight, and canonical serialization

mdstats development specification

2026-07-20

Contents

Status and authority

This specification defines and records the implementation of architecture gate **LD1-B** under the normative `mdstats Dynamical Framework and Density Plotting Architecture Standard`. It is implemented in `mdstats 0.19.46a0` from the `mdstats 0.19.45a0` LD1-A baseline, where LD1-A already provides deterministic sparse CIC aggregation and a flat-node canonical-convolution oracle.

The dense backend remains the default. The production local-sparse backend is explicit opt-in and is supported only with `discrete_periodized_v1`.

Objective

Deliver production atomic density storage and preparation on a globally defined periodic logical-node lattice without allocating a full dense scalar field:

$$\text{registered atomic samples} \to \text{sparse CIC masses} \to \text{canonical stencil scatter} \to \text{deterministic block packing} \to \text{PeriodicBlockScalarField3D}\,.$$

The scientific estimator must remain exactly the LD1-A estimator. Block packing changes representation only.

Non-objectives

LD1-B does not implement:

1. sparse probability-shell rendering;
2. sparse marching cubes;
3. framework-vertex or framework-edge sparse fields;
4. automatic dense/sparse backend selection;
5. multilevel refinement;
6. compiled or parallel accumulation kernels.

Rendering a local-sparse field is rejected until LD2-A/LD2-B.

Scientific invariants

Logical-node convention

A node with integer index (i, j, k) is located at

$$\mathbf{f}_{ijk} = \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3}\right), \qquad \mathbf{x}_{ijk} = \mathbf{f}_{ijk} H.$$

All indices are periodic modulo (N_1, N_2, N_3).

Estimator identity

The packed field must match the LD1-A flat-node reference exactly within the normative floating-point policy. Packing must not reorder scientific accumulation.

Measure normalization

For selected atomic occupancy M,

$$\Delta V \sum_g \rho_g = M, \qquad \Delta V = \frac{|\det H|}{N_1 N_2 N_3}.$$

Determinism

For fixed inputs and options, active block indices, block values, masks, public node iteration, metadata, and JSON serialization are independent of hash order.

Data model

```
@dataclass(frozen=True, slots=True)
class PeriodicBlockScalarField3D:
    schema_version: str
    field_key: str
    label: str
    physical_units: str
    logical_grid_shape: tuple[int, int, int]
    block_shape: tuple[int, int, int]
    active_block_indices: NDArray[np.int64]       # (n_blocks, 3)
    block_values: NDArray[np.float64]             # (n_blocks, bx, by, bz)
    block_valid_masks: NDArray[np.bool_] | None   # same block axes
    display_cell: NDArray[np.float64]
    total_measure: float
    gaussian_bandwidth: float
    smoothing_operator: Literal["discrete_periodized_v1"]
    broadening_metric: str
    storage_backend: Literal["local_sparse"]
    source_provenance: DensitySourceProvenance
    selected_atom_indices: tuple[int, ...]
    sample_positions: NDArray[np.float64] | None
    metadata: FrozenJSONMapping
```

All arrays are defensive, C-contiguous, and read-only. Active block indices are unique and lexicographically sorted.

Block mapping

For logical node $\mathbf{n} = (i, j, k)$ and block shape $\mathbf{B} = (B_1, B_2, B_3)$,

$$\mathbf{b} = \left\lfloor \frac{\mathbf{n}}{\mathbf{B}} \right\rfloor, \qquad \mathbf{l} = \mathbf{n} - \mathbf{b} \odot \mathbf{B}.$$

Only blocks containing positive field values are allocated. The block lattice is

$$L_a = \left\lceil \frac{N_a}{B_a} \right\rceil.$$

If any N_a is not divisible by B_a, explicit valid masks mark logical nodes inside the global grid. Invalid terminal-block slots are zero and never appear through public node iteration.

Transactional packing

Packing uses two bounded phases.

Index preflight

Before allocating `block_values`:

1. map active flat indices to logical coordinates;
2. derive unique lexicographic block coordinates;
3. compute exact block count and allocated scalar slots;
4. compute exact valid-slot count and mask requirement;
5. estimate package-owned planning workspace;
6. enforce block, slot, nonzero-node, and planning-byte limits.

Realization

After approval, allocate block arrays once and place values in ascending active flat-index order. Realized storage is checked against the approved counts.

Public node access

`gather_node_values(indices)` wraps indices periodically and returns zero for unallocated nodes.

`iter_stored_nodes()` yields positive logical nodes in global lexicographic order, not block-major order. This makes the public scientific iteration independent of block shape.

Atomic preparation

`prepare_atomic_density_fields` supports:

```
DensityStorageOptions(
    grid_backend="local_sparse",
    local_block_shape=(16, 16, 16),
)
```

with the following requirements:

- `smoothing_operator == "discrete_periodized_v1"`;
- atom-index and species selections retain existing semantics;
- the requested logical resolution is resolved independently of the dense voxel budget;
- CIC aggregation and stencil scatter use the LD1-A reference functions;
- the reference result is immediately packed into the production block field;
- sample positions are retained only when requested;
- metadata records logical, nonzero, block, valid-slot, allocated-slot, and normalization counts.

The dense default path remains unchanged.

Serialization

`to_json_dict()` emits schema-versioned scientific metadata and block arrays. `from_json_dict()` validates all dimensions, ordering, masks, storage identifiers, and normalization. Canonical JSON round trips must preserve all fields exactly.

Resource limits

LD1-B enforces:

```
max_density_nonzero_nodes
max_density_stored_block_values
max_density_blocks
max_density_kernel_pairs
max_density_planning_bytes
max_density_total_peak_bytes
```

The direct preparation API exposes equivalent optional limits. A failure occurs before `block_values` allocation whenever a packing limit is exceeded.

Acceptance criteria

Numerical equivalence

Against LD1-A for all required atomic fixtures:

```
relative L1 field error <= 2e-11
relative L-infinity field error <= 5e-11
absolute integral error <= 5e-13 * max(1, total_measure)
HDR threshold absolute difference <= 5e-12 * max(1, reference maximum)
achieved HDR mass-fraction difference <= 5e-13
```

Storage and access

1. active blocks are unique and lexicographically sorted;
2. public positive-node iteration equals LD1-A indices and values;
3. periodic gather equals the dense oracle;
4. partial-block masks are exact;
5. serialization round trips exactly;
6. block-order permutation is either rejected or canonicalized deterministically;
7. every resource limit has a focused pre-allocation failure test.

Atomic integration

1. species and explicit-index selections produce correct provenance;
2. explicit `local_sparse` preparation returns `PeriodicBlockScalarField3D`;
3. `legacy_spectral_v1 + local_sparse` is rejected;
4. framework sparse requests remain rejected until LD3;
5. scene rendering of sparse fields is rejected until LD2 rather than failing by private attribute access;
6. the dense default is byte-for-byte compatible with `0.19.45a0`.

Resource benchmark

A localized LTA-like atomic fixture must satisfy:

```
allocated block scalar slots / logical dense scalar slots <= 0.10
logical dense scalar slots / allocated block scalar slots >= 10
```

without changing the requested logical grid or Gaussian bandwidth.

Stop conditions

Do not proceed to LD2-A if:

- block packing changes LD1-A values;
- partial masks omit valid nodes or expose invalid nodes;

- public access depends on block insertion order;
- sparse resolution still inherits the dense allocation cap;
- serialization loses provenance or scientific metadata;
- the localized benchmark misses the storage gate.

Citations

Periodic trilinear cloud-in-cell assignment follows Hockney and Eastwood, *Computer Simulation Using Particles* (1988). The block-structured organization is inspired by Berger and Colella, "Local Adaptive Mesh Refinement for Shock Hydrodynamics," *Journal of Computational Physics* **82** (1989), 64-84. The periodic block ownership, exact partial masks, deterministic packing, and atomic integration policies are project-specific mdstats definitions.